The MIT Press

PWMC, a Constraint-Solving System for Generating Music Scores
Author(s): Örjan Sandred
Source: *Computer Music Journal,* Vol. 34, No. 2 (Summer 2010), pp. 8-24
Published by: The MIT Press
Stable URL: https://www.jstor.org/stable/40731279
Accessed: 09-11-2026 23:44 UTC

REFERENCES
Linked references are available on JSTOR for this article:
https://www.jstor.org/stable/40731279?seq=1&cid=pdf-reference#references_tab_contents
You may need to log in to JSTOR to access the linked references.

JSTOR is a not-for-profit service that helps scholars, researchers, and students discover, use, and build upon a wide range of content in a trusted digital archive. We use information technology and tools to increase productivity and facilitate new forms of scholarship. For more information about JSTOR, please contact support@jstor.org.

Your use of the JSTOR archive indicates your acceptance of the Terms & Conditions of Use, available at https://about.jstor.org/terms

JSTOR

The MIT Press is collaborating with JSTOR to digitize, preserve and extend access to *Computer Music Journal*

This content downloaded from 142.104.113.73 on Mon, 09 Nov 2026 23:44:09 UTC
All use subject to https://about.jstor.org/terms

Örjan Sandred
Studio FLAT
University of Manitoba
Marcel A. Desautels Faculty of Music
65 Dafoe Road
Winnipeg, MB R3T 2N2 Canada
sandred@cc.umanitoba.ca
http://umanitoba.ca/studio-flat

PWMC, a Constraint-Solving System for Generating Music Scores

This article outlines the design of the PatchWork Musical Constraint system (PWMC) for solving musical constraint-satisfaction problems. After a short introduction, the objectives and basic concepts of PWMC are presented; a short description of the system from the perspective of the user interface then follows. Next, the structure of the domain and how the user designs rules are explained in more detail. A patch example that summarizes some possibilities of the system is examined, and then the internal data representation is outlined. To conclude, we discuss a musical example.

Rule Formalization and Music

Using rules to explain music structures is not new. Music textbooks have used rules for several hundred years to clarify how pitches can be structured harmonically and melodically. Rules allow music theorists to break down a complex music description to simple sub-descriptions. Composers have also been attracted to rules in formalizing their methods of composition. Several Renaissance composers (e.g., Johannes Ockeghem, 1410–1497) used canon techniques to create complex relations among voices in polyphonic music. More modern examples can be found in serial music: The pitch row of a serial composition is a rule in itself.

Computers allow composers to experiment more deeply with how musical rules interact together. The first examples of using rules in computer music can be found in Lejaren Hiller and Leonard Isaacson's *Illiac Suite* (1958). In the first, second, and third movements, rules were used to filter out an acceptable pitch structure from randomly generated pitches. (In the third movement, durations and dynamics were also generated.) Hiller and Isaacson relied on the Monte Carlo method, where the program generated values at random until one passed the screening rules.

Musical problems that are more complex need to be solved using a more systematic method than the Monte Carlo method, however. For this reason, several constraint-solving systems for music have been developed. A pioneering example from the early 1990s is the composition environment Carla (Courtot 1990). Another early example is the combination of Backtalk, an extension to Smalltalk, and MusES (Pachet and Roy 1995).

Contemporary Constraint Solvers in Music

Today, a few constraint-solving systems with built-in support for musical structures are available; three are mentioned here. The Situation system (Bonnet and Rueda 1999) is implemented inside the visual programming language OpenMusic (Assayag et al. 1999) and is specialized in assembling harmonic structures. Chord structures are searched for independently of rhythm. Situation also gives a limited support for building rhythmical structures (independently of pitch).

Score-PMC is based on the PMC constraint solver (Laurson 1996) and exists inside the visual programming language PWGL (Laurson, Kuuskankare, and Norilo 2009). Only the pitch structure can be constrained in Score-PMC; the rhythmic structure is always given by the user. Nevertheless, rules applied to the pitches can constrain relations between pitches and arbitrary aspects of the given rhythmical structure.

Strasheela (Anders 2007) is a musical constraint–solving system based on Oz, a powerful programming language for intelligent applications. The choice of Oz makes Strasheela the most efficient constraint-based music composition system of those listed here. At the same time, Strasheela uses a text-based syntax, and the user must have a deeper

Computer Music Journal, 34:2, pp. 8–24, Summer 2010
© 2010 Massachusetts Institute of Technology.

This content downloaded from 142.104.113.73 on Mon, 09 Nov 2026 23:44:09 UTC
All use subject to https://about.jstor.org/terms

knowledge of the programming language than in the other two systems. To do Strasheela justice, its full power is best demonstrated on highly complex constraint problems where rhythmic, contrapuntal, harmonic, and motivic structures are all simultaneously sought. Strasheela can solve musical constraint–satisfaction problems for any attribute of musical events.

There are also other options for composers who wish to use constraint-solving techniques. Some composers have used constraint solvers that do not come with support for music structures—for example, Screamer, an extension to Common Lisp (Siskind 1991). A quite different type of system is OMClouds (Truchet and Codognet 2004). This system relies on a heuristic search strategy, which makes it incompatible with the other systems discussed here.

PWMC: Objectives

PatchWork Musical Constraints (PWMC) is an extension to the visual programming language PWGL, an acronym for PatchWork OpenGL. The main objective of developing PWMC was to create a system that could solve constraint-satisfaction problems for polyphonic music structures where pitches, rhythm, and meter are all unknown. The other choice for a constraint-solving system that resides inside PWGL is the Score-PMC system (mentioned earlier); however, Score-PMC can search only for a pitch structure: The rhythms and the metric structure must be known before the search begins.

PWMC was developed to support the creation of some compositions by the author (*Whirl of Leaves* for flute and harp, composed in 2006, being the first composition that fully utilized PWMC). The esthetics linked to these compositions has had an impact on PWMC; however, it has also been a concern to make the system usable for other composers and as far as possible to avoid narrow esthetic limitations.

An important goal when designing PWMC was to create a user interface that is simple to use. It should be easy for a composer to study the musical effect of tweaking and experimenting with different sets of rules. PWMC uses the visual programming language inside PWGL and has a more developed graphical interface than the systems mentioned previously. A graphical interface illustrates the modular nature of the system well: Rules can easily be enabled or disabled via patch cords. The behavior of rules can also easily be changed via menu settings.

The user interface should allow the user to design new rules. The choice of PWGL gives the user access to all previously developed functions inside this environment (as part of the PWGL kernel or as part of a PWGL library). Any existing function in PWGL can potentially be used inside a rule definition.

Less important when developing PWMC were optimization issues of the constraint-solving techniques used by the system. It was therefore decided to base the system on existing constraint solvers. The version of PWMC that is distributed on-line supports the constraint solver PMC inside PWGL; however, during the early development stages of the system, it also had support for the Situation and Screamer constraint solvers. Support for these two solvers was later removed, because they did not provide any advantages to the PWMC system.

Basic Concepts

PWMC is based on typical constraint-programming techniques for musical applications: A set of *rules* is applied to a sequence of *variables*. A variable can take any value from a *domain* of *candidates* as long as the value is consistent with all rules. A solution to a constraint-satisfaction problem is found when all variables in the sequence have been assigned satisfactory values. The process of finding an acceptable sequence is called *searching*.

In a musical constraint–satisfaction problem, the values in the domain (and in the solution) have a musical interpretation. PWMC uses three different types of candidates in the domain: pitches, durations, and metric units. A candidate can also be a group of pitches or durations. In PWMC terminology, pitches are defined in the *pitch domain*, durations in the *rhythm domain*, and metric units in the *metric domain*. The domain known by the constraint

This content downloaded from 142.104.113.73 on Mon, 09 Nov 2026 23:44:09 UTC
All use subject to https://about.jstor.org/terms

solver is the union of all the sub-domains. A variable will be assigned a value only from one of the sub-domains. A note with a pitch and duration will thus take up two variables in the solution.

Pitches are defined as MIDI note numbers. (Numbers with decimal fractions such as 60.25 will be understood as microtonal pitches.) Ratios are used to define durations: a quarter note is defined as the ratio 1/4, a dotted quarter note is 3/8, a quarter-note triplet is 1/6, etc. Negative ratios represent rests. In PWMC, a metric unit is a measure with a specified time signature. Metric units are defined as number pairs. For example, a measure with the time signature 4/4 will be defined as the number pair (4 4). Pitches, durations, and metric units in the domain are distinguished by their data types. (See the Data Representation section.)

While the events in all voices are placed inside the same sequence of metric units, pitches and durations belong to a voice: PWMC lets the user specify different domains for different voices. For example, a user can specify that the duration candidates 1/4, 1/8, and 3/8 belong to the first voice, and candidates 1/16 and 1/8 belong to the second voice. As a consequence, the note duration 1/16 cannot exist in the first voice, and 1/4 and 3/8 cannot exist in the second voice.

PWMC builds a score by combining pitches, durations, and metric units from the domains according to rules. Besides defining the number of voices, the user sets the total number of variables the score should contain. These variables will be distributed among pitches, durations, and metric units during the search process. For example, if the user asks for 100 variables in a score with two voices, the search process could result in a score with 4 metric units, 22 pitches, and 22 durations in the first voice, and 26 pitches and 26 durations in the second voice (i.e., $4 + 22 + 22 + 26 + 26 = 100$). The distribution depends on the rules.

As explained previously, PWMC uses an existing constraint solver with a flat data structure. The complex data structure that a musical score represents must be broken down into a single stream of elements where pitches, durations, and metric units are added one by one to the solution. The order of the score elements within the flat data structure will determine if, for example, the system should search for a note in the first voice before a note in the second voice, if the system should search for durations before pitches, etc. The order is crucial for computing efficiency; however, it is difficult to know what a good order is before the search has started.

The exact search order in PWMC is determined by a special group of rules called *strategy rules*. Because the rules are checked at each step in the search process, the search order will be context-dependent. A typical example is that if one voice turns out to contain many short durations, and another voice turns out to contain longer durations, the strategy rules might be set to allow the voice with shorter durations to search for more events to catch up with the other voice. (See the Strategy Rules section.)

The User Interface

PWMC uses the visual programming interface inside PWGL. The user defines the domains, creates the rules, and accesses the resulting score inside a patch. An example of a basic setup can be seen in Figure 1. Here, the system uses the PMC constraint solver—the box is called *Multi-PMC* in the figure—that is part of PWGL. The entire data flow is not visible in the patch, because the rules are applied to the individual candidates inside the constraint solver. The constraint solver receives the domain and the set of rules to carry out the search process.

The system is set to search for 40 variables. (This number can be changed in the *build-domain* box.) The example has only one voice, for which the system will search for both durations and pitches. A domain can be defined either as a list of values or algorithmically. In this example, the rhythm domain contains a sixteenth note, an eighth note, and a dotted eighth note (see the *text-box* at the top-left corner). The possible pitches are defined as an arithmetic series from 60 to 84 with an increment step of 1, which will give a list of the MIDI note numbers for all chromatic pitches from C4 to C6 (see the top-right corner). Each duration and pitch candidate is wrapped in a sub-list, which will be explained in the subsequent Motifs and Groupings of Values section.

This content downloaded from 142.104.113.73 on Mon, 09 Nov 2026 23:44:09 UTC
All use subject to https://about.jstor.org/terms

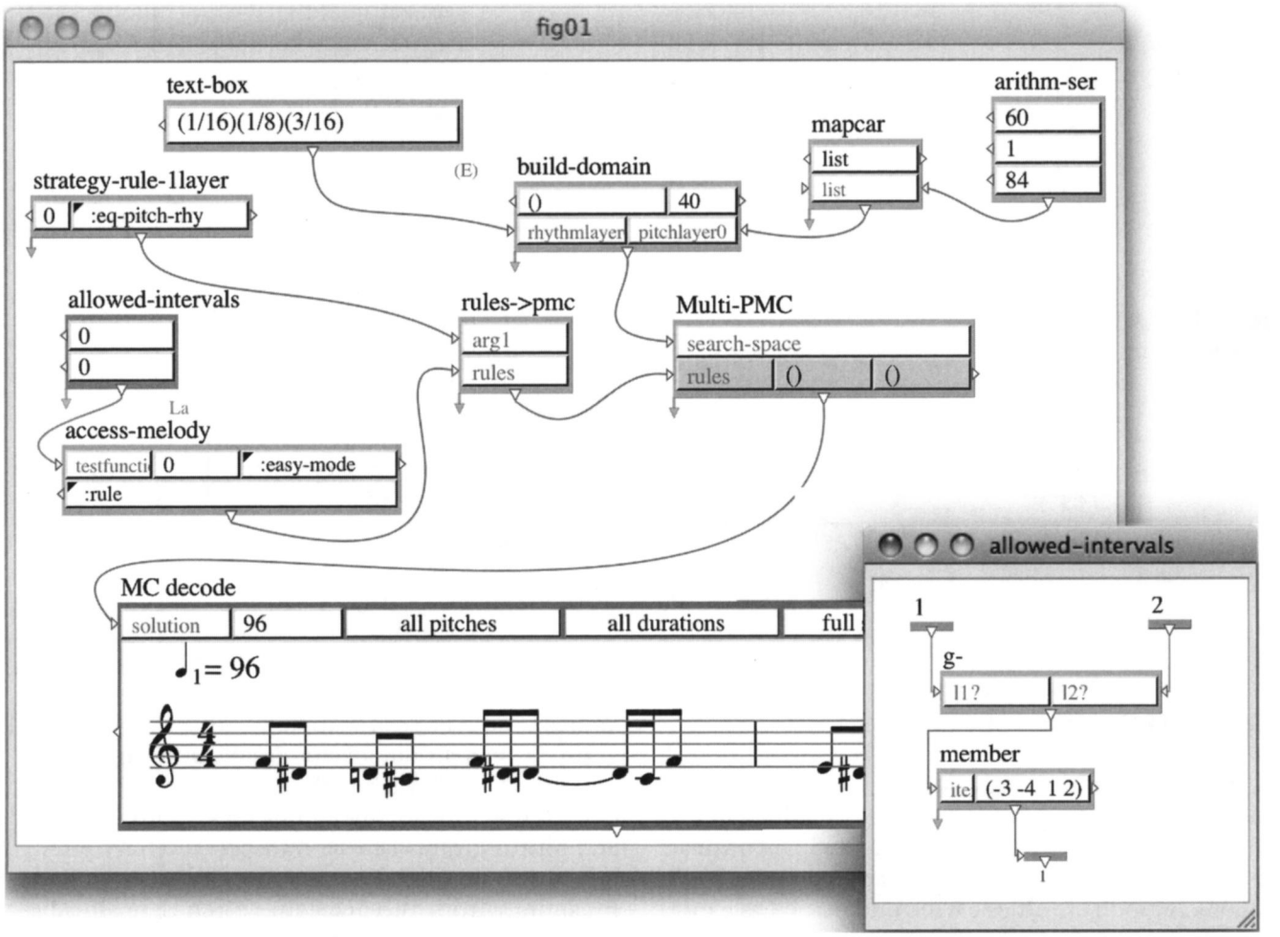


Figure 1. Basic example of the graphical interface of PWMC. The patch illustrates the elements of the system: the constraint solver (the Multi-PMC *box), the domain (the* build-domain *box and everything connected to it), the rules (the* rules->pmc *box and everything connected to it), and the solution (the* MC decode *box).*

The rules are collected by the *rules->pmc* box. Two rules are used in the example. The predefined *strategy-rule-1layer* will force the solution to have an equal number of pitches and durations; i.e., the voice will be built of 20 pitches and 20 durations. The second rule is defined by the user: the *access-melody* box will pass consecutive pitches from the voice to the inputs of the rule. The rule itself is defined inside a sub-patch. (The window of the open sub-patch can be seen in the lower-right corner in Figure 1.) The rule calculates the difference between two consecutive pitches and checks if it is equal to –3, –4, 1, or 2. This rule will allow only upwards minor or major thirds, or downwards minor or major seconds as melodic intervals.

The solution can be seen in the score at the bottom of Figure 1. The tempo, set here to MM = 96, never affects or is affected by the search process. It is used only to control the speed of MIDI playback. The tempo is set in the second input in the MC decode box.

More About the Domain

Pitches, durations, and metric units that are used by the system must exist in the domain. The following sections give a detailed explanation of how they are defined.

This content downloaded from 142.104.113.73 on Mon, 09 Nov 2026 23:44:09 UTC
All use subject to https://about.jstor.org/terms

Figure 2. An example of groupings of values in the domain.

Figure 3. This example combines groups from Figure 2. Pitch groups are indicated above the staff, rhythm groups below the staff.

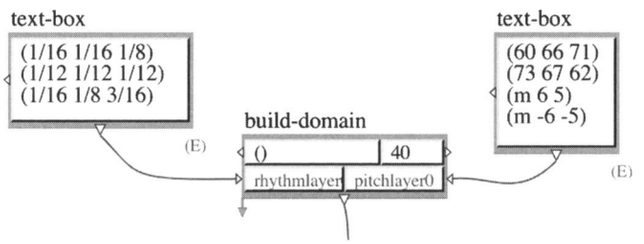


Figure 2.

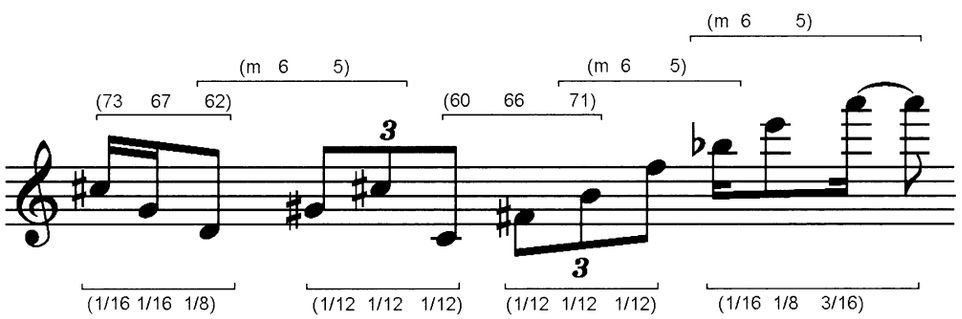


Figure 3.

Motif and Groupings of Values

One basic idea in the development of PWMC was to allow the user to define groupings of values in the domain. This idea originates from the OpenMusic Rhythmic Constraint system (Sandred 2000, 2003, 2006). A group of values can be understood as a motif or a fragment of a musical gesture. A group that is defined in the domain cannot be broken down or changed by the constraint solver. It can have any length and will occupy one variable in the solution. If only groups are used in the domain, the solution will be a combination of the groups.

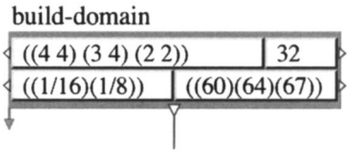
Examples of groups of values can be seen in Figure 2. The rhythm domain in this example contains three groups: two sixteenth notes followed by one eighth note, a triplet of eighth notes, and finally a sixteenth note followed by an eighth note followed by a dotted eighth note. The pitch domain contains four groups. The values in the first two groups should be read as MIDI note numbers: The first group is C4, F-sharp4, and B4, and the second group is C-sharp5, G4, and D4. The third and fourth groups begin with the letter *m* to indicate that their values should be read as melodic intervals. The third group is thus a sequence of an upwards augmented fourth followed by a fourth. The fourth group is built of the same intervals but going downward. The third and fourth groups can be transposed to any pitch, but not in an arbitrary way: a group of intervals must start from the preceding pitch that already exists in the score. Groups of melodic intervals will thus overlap, whereas groups of absolute MIDI note values will not. The pitch sequence cannot start with a group of melodic intervals, because the starting pitch would be unknown. A possible combination of the groups in Figure 2 can be seen in Figure 3.

Metric Units

Many rules that can be found in traditional music theory restrict pitches in relation to the metric hierarchy. Typically, the treatment of consonance and dissonance relate to an event's metric position. PWMC understands the concept of metric beats; for example, even though the time signatures 4/4 and 2/2 create measures with the same lengths, the former will contain four beats and the latter

This content downloaded from 142.104.113.73 on Mon, 09 Nov 2026 23:44:09 UTC
All use subject to https://about.jstor.org/terms

Figure 4. A metric domain with three different metric units: 4/4, 3/4, and 2/2.



two beats. Rules that refer to the metric structure might give different solutions for the different beat structures that these two time signatures have.

Metric units (i.e., measures with a specific time signature) are defined in the domain, and PWMC can search for a sequence of acceptable metric units for pitches and rhythm (see Figure 4). The score representation in PWMC does not allow different time signatures in different voices. Instead of metric complexity, the main focus when developing and using PWMC has been to explore the musical relationship among pitch, rhythm, and meter.

Locking Variables

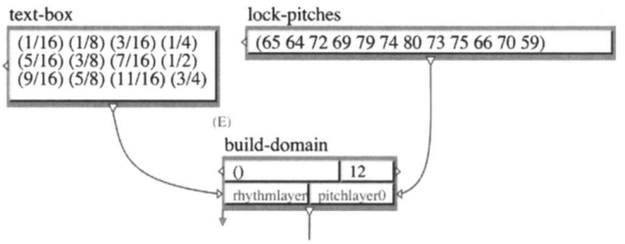
The *build-domain* box also allows the user to input a sequence of pitches or durations that cannot be changed by the constraint solver. The idea here is that the user manually composes certain parts of the solution (e.g., the durations or pitches of a certain voice). The system will then fill in whatever is missing.

In Figure 5, a sequence of twelve pitches (here the twelve-tone series from the first movement in Alban Berg's *Lyrische Suite*) is predefined for the first voice in the domain. This is done by entering the sequence into the box *lock-pitches* and connecting that box's output to the appropriate input on the *build-domain* box. The *lock-pitches* box will tell the system that the sequence is not part of the domain, but it will be known by the system. Any rule can refer to this sequence during the search process. The system will search for durations to complement the predefined pitch sequence. The rhythm domain contains twelve possible durations that can be used for the rhythm.

Defining Rules

A few rules in PWMC are predefined (e.g., a rule for creating a rhythmic canon between voices). More

Figure 5. A predefined sequence of pitches connected to the build-domain *box.*



typically, the user designs new rules using PWGL's visual programming interface. A rule definition in PWMC has two parts: a sub-patch with a logic statement, and an access box. The access box passes pitches, durations, and/or metric information from the score to the sub-patch. The sub-patch applies its logic statement to the values: if the result is true, the pitches, durations, and/or metric information passed the rule test.

Formally, the sub-patch with the logic statement is a function in PWMC. The access box adds further arguments (that describe what elements in the score the rule should restrict) to the function. In this way, the rule that the access box passes to the constraint solver will contain both the rule restriction and information on how the constraint solver can access the restricted values from the score.

This method for designing rules is a development of the more primitive Open Music Rhythmic Constraint system (OMRC; Sandred 2000). In OMRC, there is also a division between the rules (that are passed as functions) and a box that adds information for what voice each rule restricts. The design of passing a logic statement as a function and adding additional arguments for how to access score parameters is also proposed in Strasheela (Anders 2007).

Access Boxes

PWMC provides several access-boxes from which to choose. Recall that in Figure 1, the *access-melody* box passed notes to a sub-patch that constrained melodic intervals. Figure 6 shows the most important access boxes in PWMC. All access boxes expect a sub-patch with a logic statement connected to the input in the top-left corner. The number of the voice/voices the rule restricts is set in the following one or two inputs. (However,

This content downloaded from 142.104.113.73 on Mon, 09 Nov 2026 23:44:09 UTC
All use subject to https://about.jstor.org/terms

Figure 6. The most important access boxes in PWMC. The sub-patches with the logic statements are missing (and that is why the top-left input on all boxes show an empty list).

Figure 7. Menu inputs on the access-2part-harmony *and* access-metric-structure *boxes. (The input options are unfolded and visible on both boxes.)*

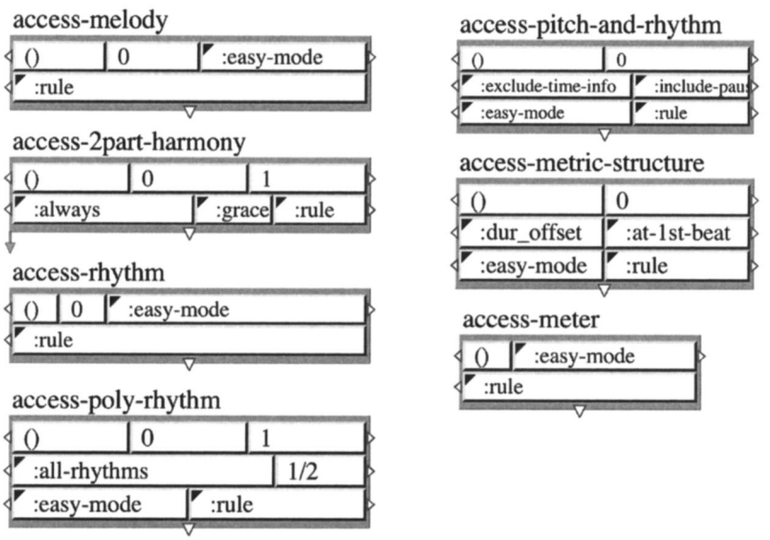


Figure 6.

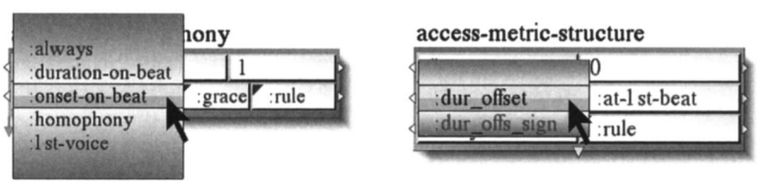


Figure 7.

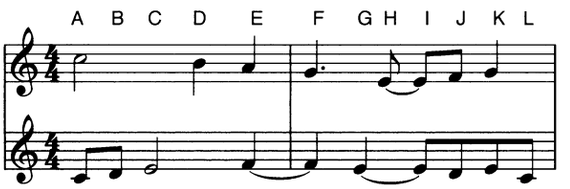
access-meter is an exception, because it passes only metric units without accessing voices.) All boxes except *access-2part-harmony*, *access-poly-rhythm*, and *access-meter* will accept a list of voice numbers. The same rule will then be applied to all voices in the list.

All access boxes have additional inputs where settings can be selected with menu inputs. Two examples of menu options on access boxes can be seen in Figure 7. The left example shows the *access-2part-harmony* box. This access box will pass pairs of simultaneous pitches belonging to the two voices indicated in the second and third input in the top row of the box. (If the sub-patch with the logic statement has more than one input, the inputs will be understood as consecutive pitch pairs.) The first menu in this box gives five options for how the pitch pairs will be accessed: *always* indicates that every existing pair of simultaneous pitches between the two voices will be checked by the rule; *duration-on-beat* indicates that only simultaneous pitches that exist on beats will be checked by the rule, and any off-beat pitch pair will be ignored; *onset-on-beat* is a special case of the previous option, in which events need to have their starting point at a beat to be checked by the rule; *homophony* indicates that only pitches that have simultaneous starting points will be checked by the rule (the metric positions of the events are irrelevant for this option); and *1st-voice* indicates that the rule will only be checked at starting points for the first of the two voices (i.e., the voice indicated in the second input in the first row).

Figure 8 illustrates the effect of the settings of the first menu on the *access-2part-harmony* box. If *always* is selected, every pitch pair (A–L) will be accessed. If *duration-on-beat* is selected, then A, C, D, E, F, G, I, and K will be accessed. If *onset-on-beat*

This content downloaded from 142.104.113.73 on Mon, 09 Nov 2026 23:44:09 UTC
All use subject to https://about.jstor.org/terms

Figure 8. The letters refer to the harmonic intervals between the two voices (one letter for each rhythmic event).



is selected, then A, C, D, E, F, G, and K will be accessed. If *homophony* is selected, then A, E, J, and K will be checked. If *1st-voice* is selected, then A, D, E, F, H, J, and K will be accessed. Even though the *access-2part-harmony* box passes only pitch information to the logic statement, the rule will also have an impact on rhythm, because the constellation of pitch pairs depends on rhythm.

The right side Figure 7 shows the *access-metric-structure* box. This access box provides the logic statement with information about events that occur on beats. Any event that does not occur at the point of a beat will be ignored. The idea is to make it possible to restrict the type of syncopations that can exist in a score. The most important information the box passes to its logic statement is the event's offset from the beat. If there is an event that starts on a beat, its offset will be zero. If an event starts before a beat (and is sustained over the point of the beat), the offset will indicate how long before the beat the event starts. For example, an offset of –1/8 indicates that the event starts an eighth note before the beat. Events that do not start on a beat and are not sustained over the point of a beat will be ignored.

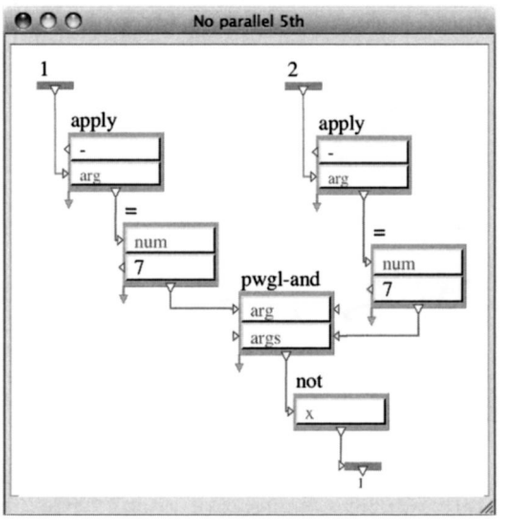
There are two options for the first menu in the *access-metric-structure* box: Either the box can be set to pass a list consisting of the duration of the event and its offset to the logic statement, or it can be set to pass a list consisting of the duration of the event, its offset, and the time signature of the measure.

The current version of the software still is missing some access boxes for certain ways of accessing score information. Access boxes were developed and added to PWMC when they were needed by the developer.

Logic Statements

A logic statement is defined inside a sub-patch. The user creates the logic statement by using the visual programming interface inside PWGL. The parameters that the logic statement will receive depend on which access box the sub-patch is connected to and on the settings in the access box.

Figure 9. A sub-patch with a logic statement that forbids parallel fifths between voices.

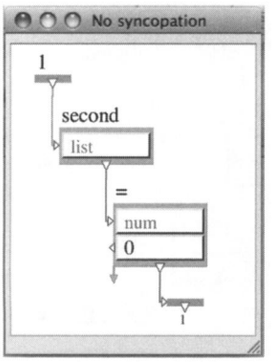


The logic statement in Figure 9 is an example of a rule that forbids parallel fifths between voices. The output of the sub-patch should be connected to the *access-2part-harmony* box. As implied by the earlier discussion of the this box, input 1 of the sub-patch in Figure 9 will receive one pair of simultaneous pitches, and input 2 will receive the next pair. The logic statement compares two consecutive pitch pairs and makes sure that at least one is not a fifth. Depending on the settings on the access box, the parallel fifths will be restricted between all harmonic intervals or at selected points (see the previous Access Boxes section).

Another example of a logic statement can be found in Figure 10. This sub-patch should be connected to the *access-metric-structure* access box. The box should be set to receive a list of a duration and its offset and to restrict only the first beat in every measure. (This is selected in the access box's second menu input; see Figure 6.) The logic statement only uses the value for the offset from the input and states that the offset must always be zero. This rule will allow measures to start only with an event on the beat. As a consequence, syncopations over bar lines will not be allowed.

This content downloaded from 142.104.113.73 on Mon, 09 Nov 2026 23:44:09 UTC
All use subject to https://about.jstor.org/terms

Figure 10. A logic statement that forbids syncopations over bar lines.



Heuristic Rules

Heuristic rules do not use logic statements to evaluate candidates. Instead of giving "true" or "false" as the result, a heuristic rule will sort the candidates in order from the most preferred to the least preferred; the sub-patch must be designed to produce a numerical weight instead of a Boolean value. The candidate that receives the highest weight is the most preferred candidate.

All access boxes in PWMC can be used to create heuristic rules. The bottom-right menu input can be set to either *rule* or *heuristic-rule*. An example of the use of a heuristic rule can be seen in the musical example at the end of this article.

A Patch Example

An example to illustrate the PWMC system follows. The example is chosen not for its musical value but because it combines several of the ideas presented herein. The idea of this example is to apply serial control to both pitches and durations in two voices. The pitch series for the first voice is already given (the twelve-tone row from the first movement in Alban Berg's *Lyrische Suite*, mentioned earlier in connection with Figure 5). The system will search for the pitch series for the second voice. The durations in both voices will be built of twelve different durations: No duration can be used more than once.

The patch for the example is shown in Figure 11. The pitch row from *Lyrische Suite* is given inside the *lock-pitches* box. The pitch domain for the second voice consists of all pitches between MIDI note numbers 48–69 (i.e., C3–A4). The durations in the rhythm domain are organized in five groups that can be understood as five rhythmic motifs. The five groups of durations consist of twelve durations all together, and no duration occurs more than once. Because the text-box containing the durations is connected to both voices 1 and 2 on the *build-domain* box, it will be valid for both voices.

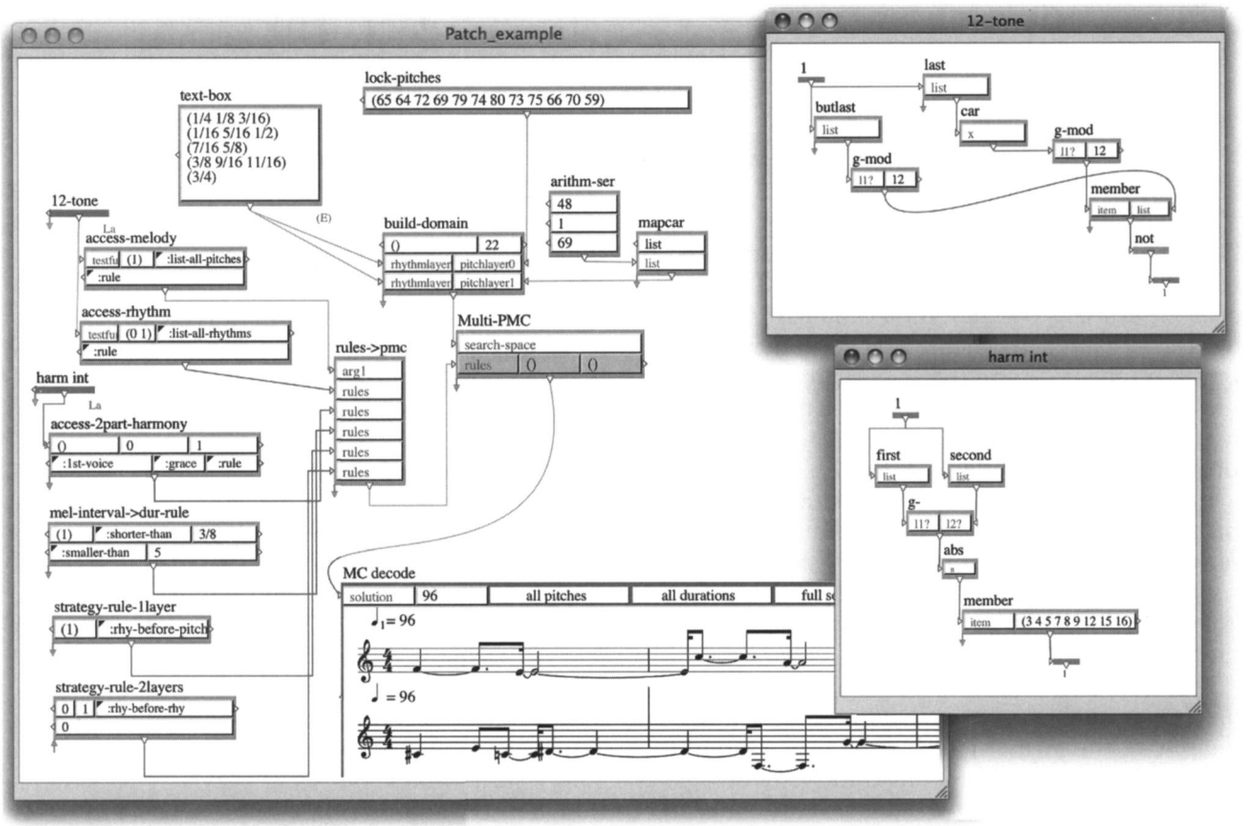
The example contains six rules. The first two rules use the same logic statement, called *12-tone*. The sub-patch for this logic statement is open in the top-right corner in Figure 11. The *access-melody* and *access-rhythm* boxes are set to pass a list of all accumulated pitches and durations, respectively, that are known at the time during the search process when the rule is checked. Variables within a single voice are always searched for in the order they appear in the score. The logic statement inside the sub-patch will check that the proposed candidate is not a member of the list of previously accepted values. In this way, the rule will ensure that all values in the list are unique. Even though the durations in the domain are grouped into motifs, the rule will check the durations one by one. Because the example will end after twelve pitches and twelve durations in each voice, these two rules will create twelve-tone rows for pitches and durations.

The third rule has a sub-patch called *harm int* connected to the *access-2part-harmony* box. This sub-patch can be seen below the twelve-tone sub-patch in Figure 11. The first menu input on the *access-2part-harmony* box is set to *1st-voice*: the rhythm in the first voice will determine where the rule will be applied (see the earlier section Access Boxes). The rule will allow harmonic intervals between the two voices only from a list of allowed intervals: minor or major third, fourth, fifth, minor or major sixth, octave, or octave plus minor or major third. This rule will not only restrict how the pitch series in the second voice can be designed, but as a side effect it might also affect the rhythm in both voices.

The fourth rule is applied only to the second voice. It restricts melodic intervals in relation to durations. In this example, notes with durations

This content downloaded from 142.104.113.73 on Mon, 09 Nov 2026 23:44:09 UTC
All use subject to https://about.jstor.org/terms

Figure 11. A patch example explained in the text.



shorter than a dotted quarter note can only continue with a melodic interval smaller than a fourth.

Finally, there are two strategy rules in the example. *Strategy-rule-1layer* states that there should be an equal number of pitches and durations in the second voice. (The rhythms should be chosen before the pitches.) *Strategy-rule-2layers* states that the two voices should have the same total duration. (A duration should be chosen for the first voice before a simultaneous duration is chosen for the second voice.) There is no musical intention behind these strategy rules; their purpose is to find an efficient search path when the system solves the problem (discussed in the next section), and to avoid solutions where the system does not create sequences of sufficient lengths.

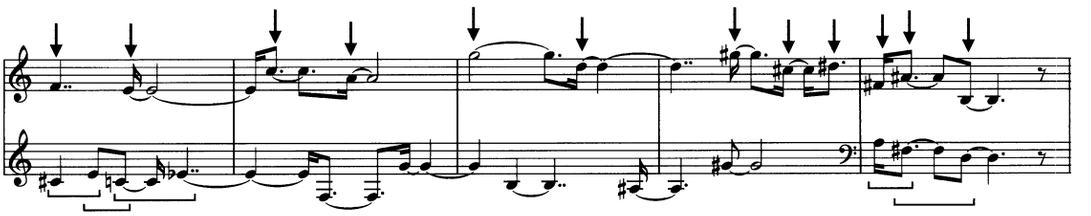
Figure 12 shows a score resulting from the patch. The arrows indicate points where the fifth rule (restricting harmonic intervals between the two voices) is applied. The brackets under the second voice indicate places where the fourth rule (the link between durations and melodic intervals) has an effect. The rhythms in both voices are built from the rhythm motifs in the domain: the first voice starts with the third group, followed by the fourth group, followed by the fifth group, followed by the first group, and finally followed by the second group.

Strategy Rules

The purpose of strategy rules is to control the order in which the variables are visited by the constraint solver. The objective is to find a variable order that is computationally efficient. Strategy rules also affect

This content downloaded from 142.104.113.73 on Mon, 09 Nov 2026 23:44:09 UTC
All use subject to https://about.jstor.org/terms

Figure 12. A score that is a result from the patch in Figure 11. This example is chosen to illustrate features rather than for its musical value. The pitch series from the first movement of Berg's Lyrische Suite *is used in the upper voice. The system searches for the pitch series for the second voice and the duration series for both voices.*



the distribution between pitches and durations (within a voice or between two voices). They can force the system to search for a candidate belonging to a certain voice and being of a certain type at a certain moment during the search process. Strategy rules, however, have no direct musical impact: Even though they control what type of candidate the constraint solver should look for (e.g., a duration in the first voice), they never restrict the value the candidate should have (e.g., what the duration should be). Although strategy rules do not have any direct musical impact, they might change the probability for finding a certain solution as the first solution.

There are two types of strategy rules: *strategy-rule-1layer* and *strategy-rule-2layers*. A typical example of the former rule restricts the system to finding a solution in which the number of durations (excluding rests) in a voice equals the number of pitches in that voice. Without this rule, the system might end up with a solution that has many pitches but few corresponding durations (or vice versa). The rule will typically force the system to alternate between searching for a pitch and a duration. An example of the latter rule restricts the total duration of two voices to be as close as possible. This rule will create a search pattern where the system alternates between two voices. (The exact time point for when the system switches from one voice to the other depends on the length of the durations in the two voices.)

Strategy rules also let the user specify that a pitch should be searched for only if there is an assigned duration for the event (or vice versa), or that events in a voice should not be searched for until another voice has events assigned at a simultaneous time point. The latter example will simplify finding solutions for certain patches in which harmonic rules are applied.

Data Representation

PWMC relies on a constraint solver with a flat data structure; a music score, however, has a more complex structure than this. An important aspect of PWMC is to provide data representation that permits the constraint solver to formalize dependencies among pitches and durations in different voices. The flat data structure the constraint solver uses is for this reason extended by PWMC's internal score representation. The user does not see the internal score representation; the solution is displayed in a traditional score, separate from the internal score representation.

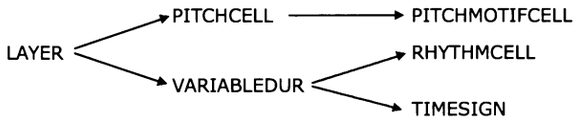
The representation of the candidates in a domain in PWMC reflects the internal score representation. The user assigns values to the candidates; the system adds information about what they are (i.e., a duration, a pitch, a metric unit) and the voice to which they belong. We now provide a more detailed description of the internal data representation in PWMC.

The Domain

The box *build-domain* (seen in Figures 1, 2, 4, 5, and 11) lets the user define separate domains for different voices as well as a domain for metric units. The domain known by the *PMC* constraint solver is the union of all the sub-domains. To enable splitting the domain known to the solver into separate sub-domains known to the PWMC system, the *build-domain* box creates objects of four different Lisp classes from candidates the user specifies: The classes are *pitchcell*, *pitchmotifcell*, *rhythmcell*, and *timesign* (see Figure 13). *Pitchcell* and *rhythmcell* objects can contain a single value or a group of values. *Pitchmotifcell* objects are defined as melodic intervals and will thus generate at least two

This content downloaded from 142.104.113.73 on Mon, 09 Nov 2026 23:44:09 UTC
All use subject to https://about.jstor.org/terms

Figure 13. Class hierarchy in PWMC.



pitches in the solution. *Timesign* objects contain a single metric unit (i.e., a time signature). All objects have a slot called *layer* that tells the system to which voice an object belongs. The metric units all belong to a separate layer. When a rule is applied, the rule is able to determine whether it should restrict the object. For example, a rule that restricts melodic intervals in the first voice will not have any direct effect on an instance of the *rhythmcell* class.

The whole set of sub-domains is used as the domain for every search variable in PWMC. All variable can thus be assigned any value from the set of sub-domains.

Score Representation

PWMC's internal data representation is based on the structure of a music score. The score is simplified, and any aspect that relates to the graphical display of the score is not part of the representation. For example, whether an off-beat quarter note should be split up into two slurred eighth notes is not indicated in the internal representation, only the total duration and the position of the event. The purpose is to make it easier for the system to handle situations when the solver needs to backtrack and reconsider earlier decision. For example, if the system backtracks and changes the time signature for a sequence of events, that quarter note might end up on a beat instead of being off-beat. The simplified score representation will not have to deal with re-notating the event in its new metric position.

PWMC uses a version of the event-list concept for the internal score representation: Several parallel sequences of values are stored. Each voice is represented as a sequence of pitches and a sequence of durations. Pitches are stored as MIDI note numbers, and durations are stored as their onset and release time. (Time is always measured as note values, i.e., as ratios.) In the sequence of durations, a release time is the next event's onset time. Release times do not therefore need to be stored separately. Rests are

Figure 14. The internal score representation in PWMC. See the text for a detailed explanation.

Variable Index Number	Rhythm Pointers	Pitch Pointers	Time Points	Number of durations (excluding rests)	Pitches
1	NIL	(1 12)	NIL	0	NIL
2	(1 3)	(1 12)	1	1	65
3	(1 3)	(1 12)	23/16	2	64
4	(3 6)	(1 12)	33/16	3	72
5	(3 6)	(1 12)	39/16	4	69
6	(3 6)	(1 12)	3	5	79
7	(3 6)	(1 12)	59/16	6	74
8	(6 7)	(1 12)	71/16	7	80
9	(6 7)	(1 12)	75/16	8	73
10	(6 7)	(1 12)	77/16	9	75
11	(7 10)	(1 12)	5	10	66

indicated with negative onsets. To make it possible to have a rest as the first event, all time points are given with an offset of unity rather than zero (an initial rest will thus have the onset –1).

The reason for storing onsets and release times rather than the durations of the events is that calculating an event's onset time from a list of durations is less efficient than calculating a duration from a list of onsets and release times. In the former case, we need to calculate the sum of all previous durations, whereas in the latter case we only need to calculate the difference between two adjacent time points. Many rules typically need to know the onset time for an event, and by storing these time points in the internal score representation, it is faster to verify these type of rules.

The table on the right in Figure 14 shows the beginning of the internal score for the first voice of the music example in Figure 12. (The internal representation of the second voice will not be included in this example.) The first column shows the onset times for the events (where an onset time also is the previous event's release time). The duration of the first event is thus 23/16 – 1 = 7/16, the duration of the second event is 33/16 – 23/16 = 5/8, etc. (Compare this to the rhythm in the first voice in Figure 12.) The second column shows the number of durations excluding rests in the sequence up to that point (including the duration itself). In this example, there are no rests, so this column only numbers the onsets for the events. The third column shows the MIDI note numbers for pitches.

This content downloaded from 142.104.113.73 on Mon, 09 Nov 2026 23:44:09 UTC
All use subject to https://about.jstor.org/terms

(Compare the MIDI note numbers with the pitches at the beginning of the first voice in Figure 12.)

It is essential that the constraint solver know the order in which parameters are added to the score. In case of backtracking, the system needs to know what part of the score should be erased and rewritten. For this reason, the index numbers of the variables are interpreted as indexes to a list of pointers to the internal score. If the constraint solver backtracks to an earlier variable, the list of pointers will indicate what part of the score should be discharged due to the backtracking and where in the score the system should continue its search.

The table on the left in Figure 14 shows the beginning of the list of pointers to the pitch and duration sequences for the first voice. The second column contains pointers for the rhythm. At the second variable (the second row of the left table), a group of two durations was added: the pointer (1 3) points at the onsets in rows 1–3 in the right table (rows in the right table are numbered from zero): 1, 23/16, and 33/16. These time points give the duration 7/16 and 5/8. At the third variable, there is no change; we assume the system added something to the second voice at this point. At the fourth variable, the system added a group of three durations (the time points 33/16, 39/16, 3 and 59/16, which give the durations 3/8, 9/16 and 11/16), and so on. In this example, the pointers in the third column in the left table always point at pitches 1–12. This is because the twelve pitches were pre-defined (see the patch example associated with Figures 11 and 12).

How a sequence of pitches matches a sequence of onsets in a single voice depends on whether rests are inserted in the sequence of durations. Each rest will offset the match of the pitch and onset sequences by one element. The second column in the right table in Figure 14 will help speed up the match of a pitch-onset pair when rests exist in a sequence.

The metric information is stored in two vectors in a similar way to rhythm. The time-points for the metric beats are stored in the first vector, and the time signatures are stored in the second vector. This format has been chosen to optimize the way in which rules find out how events relate to the metric structure: The time points for the beats can be compared with onset times for events.

The internal score is updated at each step during the search process, and it always includes all accepted candidates up to that point. The internal score representation is difficult for a user to read, so the moment the system finds a solution, a traditional music score with all necessary graphical aspects will be generated. (The ENP music notation inside PWGL is used to display the score to the user.) The user will normally not see the simplified, internal representation (and an average user does not need to know about it).

Music Example: *Labyrinths in the Wind*

The example is taken from the beginning of *Labyrinths in the Wind* for Yamaha WX5 Wind Controller and Symphony Orchestra, by the author. The piece was premiered at the Winnipeg Symphony Orchestra New Music Festival 2009 in Canada.

In this example, PWMC was used to generate the solo part; the patch can be seen in Figure 15. The structure of the orchestra part is simplified in the patch: Only the chords and their positions are used.

The *build-domain* box defines two voices, the first voice being the solo part and the second voice the orchestra part. The pitch domain of the first voice is defined as an arithmetic series from 55 to 88 (i.e., G3–E6). The rhythm domain for the first layer contains four groups of durations, two single durations, and one rest (seen in the text-box connected to the *build-domain* box in Figure 15).

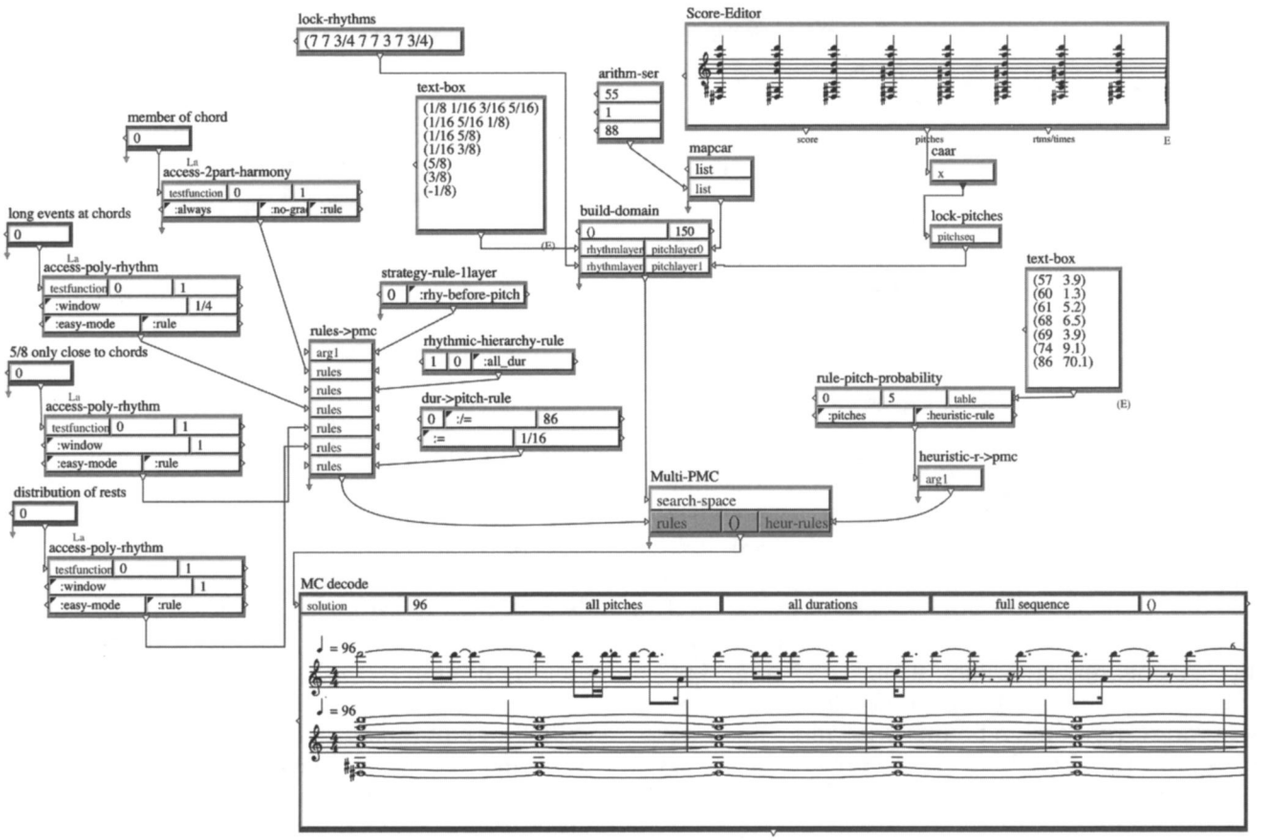
The second voice is pre-composed and cannot be changed by the system. The pitch material is a sequence of eight chords (see the upper-right corner of Figure 15). This list of chords is connected to the *build-domain* box via the *lock-pitches* box. The durations of the chord are connected via the *lock-rhythms* box. The patch contains seven rules and one heuristic rule. Four of the rules are user-defined and use access boxes that pass values to logic statements inside sub-patches.

Rules for Rhythm

The *rhythmic-hierarchy-rule* restricts the relationship between the two voices: Each time there is a new event in the lower voice, the upper voice is

This content downloaded from 142.104.113.73 on Mon, 09 Nov 2026 23:44:09 UTC
All use subject to https://about.jstor.org/terms

Figure 15. The patch for the beginning of the solo part in Labyrinths in the Wind.



forced to have a simultaneous starting point for an event. The result is that when the orchestra plays a new chord, the solo instrument will accentuate this with a new note.

Three rules employ the *access-poly-rhythm* box. The menu in the middle row on all three boxes is set to "window." This setting will make the access boxes pass segments of the rhythm from the score to the sub-patches. The sub-patch for the first of the three rules will be examined more in detail.

The first rule examines segments of a quarter-note length (see the setting next to the "window" setting on the *access-poly-rhythm* box in Figure 15). The information the sub-patch receives is

```
(((durations-voice-1) offset1)
 ((durations-voice-2)offset2))
```

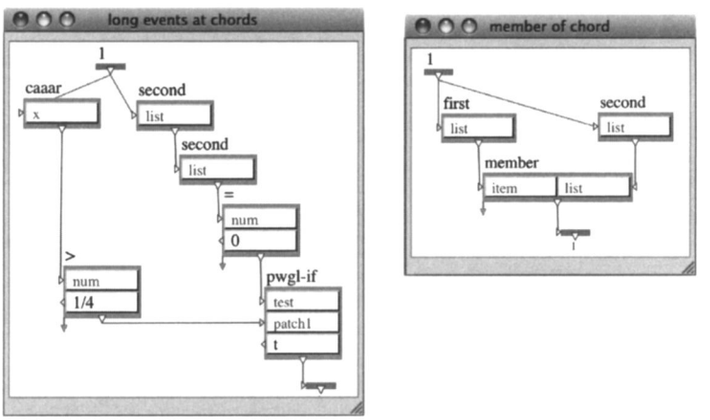
The offsets indicate if the first event coincides with the starting point of the segment (offset = 0) or starts before the segment (offset < 0). The logic statement in the sub-patch (called "long events at chords"; see the left window in Figure 16) states that if the offset in the second voice is zero, then the first duration in the segment of the first layer must be longer than a quarter note. A new chord event should thus be accentuated by a long duration in the solo voice.

The second of the three rules examines segments of a whole-note length (i.e., the length of a measure). This rule restricts the duration 5/8 to only exist within three measures after the time point for a new chord in the orchestra part. (The logic statement is not shown here.) The longest duration in the domain for the first voice is 5/8.

The third of the three rules also examines segments of a whole-note length. This rule controls

This content downloaded from 142.104.113.73 on Mon, 09 Nov 2026 23:44:09 UTC
All use subject to https://about.jstor.org/terms

Figure 16. Two of the sub-patches from the patch in Figure 15.



the distribution of rests in the solo voice. Rests are allowed during the first three measures after the time point for a new chord in the orchestra part. Between 3 and 5 measures after a new chord, there should be 1–3 more notes than rests, and between 5 and 6 measures after a new chord, there should be equal or up to two more notes than rests, and after six measures there should be equal or one more duration than rests. The result is that the solo voice will interrupt its melodic line with more and more rests until the next chord event in the orchestra part (and then the process starts all over again).

Rules for Pitch

The rule in the top-left corner in Figure 15 uses the *access-2part-harmony* box. The menu in the second row on the box is set to "always" (explained in the Access Boxes section). In this example, layer 2 is built of chords and not single pitches. The pitches for the second layer will thus be lists of the pitches in a chord. The logic statement (called "member of chord"; see the right window in Figure 16) checks that a pitch in the first layer is a member of the chord in the second layer. This rule forces the solo voice to only play pitches that are contained in the orchestra chords.

The last rule, *dur->pitch-rule*, matches durations and pitches within the first layer. The rule states that if the MIDI note number of a note is not equal to 86, the note must have the duration 1/16. The effect is that the solo voice mainly plays D6 unless sixteenth notes are encountered.

A Heuristic Stochastic Rule

Finally, there is a heuristic rule for the pitches in the first layer called *rule-pitch-probability*. Recall that heuristic rules can never fail; they can only give preference for a certain solution. The *rule-pitch-probability* receives a probability table in its top-right input (see the text-box connected to the rule in Figure 15). The values in the table are pairs of MIDI note numbers and their probabilities. The goal of this rule is to achieve a pitch sequence with a pitch distribution as close as possible to the given probability table. At each step during the search process, the rule calculates the probability distribution for the pitches in the first voice of the solution up to that point, and it compares the actual distribution to the given table. The constraint solver will prefer the candidate that gives the closest match at each step.

Examining the Solution

The beginning of the solution can be seen in Figure 17. The effect of the rules is clear. The solo voice starts with a long duration (the second rule discussed earlier). No rests are allowed during the first three measures, and in measure 4, there are three more durations than rests (the fourth rule). Only pitches from the chord exist in the solo voice (the fifth rule). Only sixteenth notes are allowed to have pitches other than D6 (the sixth rule).

This content downloaded from 142.104.113.73 on Mon, 09 Nov 2026 23:44:09 UTC
All use subject to https://about.jstor.org/terms

Figure 17. The beginning of a solution to the patch in Figure 15.

Figure 18. Measures 15–17 from the same solution as in Figure 17.

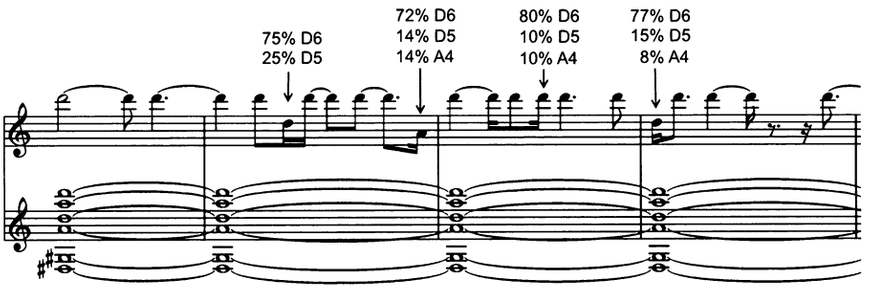


Figure 17.

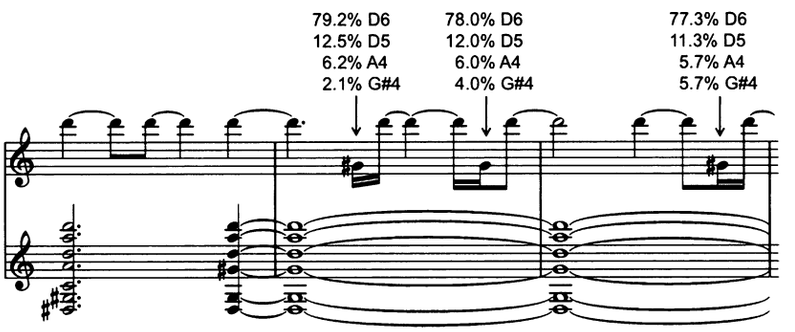


Figure 18.

The heuristic rule tries to respect the given probability table within what the strict rules allow. The given probability table states that there should be 70.1% D6, 9.1% D5, 3.9% A4, 6.5% G-sharp4, 5.2% C-sharp4, 1.3% C4, and 3.9% A3 in the solo voice. From these pitches, the first chord will only allow D6, D5, and A4 to exist. It will thus be impossible to find a precise match between the given probability table and the actual pitch distribution at the beginning of the piece. In Figure 17, the actual distribution is indicated at the points of the sixteenth notes.

Figure 18 shows the solution starting at measure 15. This is the first time in the piece where the chords allow the solo voice to use G-sharp4. The heuristic rule will now try to compensate for the lack of this pitch earlier in the piece. As can be seen in Figure 18, the probability distribution is step-by-step getting closer to the given probability table. Still, at this point in the piece, some of the pitches in the given probability table have not been allowed. This music example illustrates how heuristic rules can be of musical value in a musical constraint-satisfaction problem. For other music examples, see Sandred (2009).

Summary

PWMC is a system for solving constraint-satisfaction problems in which the solution is a musical score. A main focus while developing the system has been to create a graphical user interface that supports the composer in an intuitive way. The pitches, rhythm, and metric structure can all be unknown before the search; however, it is also possible to give the system some predefined sequences as starting points.

The system supports user-designed rules; it also allows users to structure domains in different ways, for example by defining melodic or rhythmic motifs. A limitation of the current system is computational inefficiency. Future developments of the system will

This content downloaded from 142.104.113.73 on Mon, 09 Nov 2026 23:44:09 UTC
All use subject to https://about.jstor.org/terms

look into the efficiency of the constraint solver and its search strategies.

References

Anders, T. 2007. "Composing Music by Composing Rules: Design and Usage of a Generic Music Constraint System." PhD thesis, Sonic Arts Research Centre, Queen's University, Belfast, Ireland.

Assayag, G., et al. 1999. "Computer Assisted Composition at IRCAM: From PatchWork to OpenMusic." *Computer Music Journal* 23(3):59–72.

Bonnet, A., and C. Rueda. 1999. *OpenMusic. Situation. Version 3*, third ed. Software documentation. Paris: IRCAM.

Courtot, F. 1990. "A Constraint Based Logic Program for Generating Polyphonies." *Proceedings of the 1990 International Computer Music Conference*. San Francisco, California: International Computer Music Association, pp. 292–294.

Hiller, L., and L. Isaacson. 1958."Musical Composition with a High-Speed Digital Computer." *Journal of the Audio Engineering Society* 6(3):154–160.

Laurson, M. 1996. "PATCHWORK: A Visual Programming Language and Some Musical Applications." PhD thesis, Sibelius Academy, Helsinki, Finland.

Laurson M., M. Kuuskankare, and V. Norilo. 2009. "An Overview of PWGL, a Visual Programming Environment for Music." *Computer Music Journal* 33(1):19–31.

Pachet, F., and P. Roy. 1995. "Mixing Constraints and Objects: A Case Study in Automatic Harmonization." *Proceedings of TOOLS-Europe'95*. Upper Saddle River, New Jersey: Prentice Hall, pp. 119–126.

Sandred, Ö. 2000. *OMRC Library Tutorial. Version 1.1*, second ed. Software documentation. Paris: IRCAM.

Sandred, Ö. 2003. "Searching for a Rhythmical Language." *PRISMA 01*. Milano: Euresis Edizioni, pp. 117–127.

Sandred, Ö. 2006. "*Kalejdoskop* for clarinet, viola and piano." In C. Agon, G. Assayag, and J.Bresson, ed. *The OM Composers' Book 1*. Paris: Editions Delatour France/IRCAM, pp. 223–235.

Sandred, Ö. 2009. "Approaches to Using Rules as a Composition Method." *Contemporary Music Review* 28(2):149–165.

Siskind, J. M. 1991. *Screaming Yellow Zonkers*. Software documentation. Cambridge, Massachusetts: MIT Artificial Intelligence Laboratory.

Truchet, C., and P. Codognet. 2004. "Solving Musical Constraints with Adaptive Search." *Soft Computing* 8(9):633–640.

This content downloaded from 142.104.113.73 on Mon, 09 Nov 2026 23:44:09 UTC
All use subject to https://about.jstor.org/terms